08. Feasify Comprehensive Security Assessment & Vulnerability Audit

Document ID: FEASIFY-DOC-08

Audit Standard: OWASP Top 10 Web Application Security Risks

Date: November 11, 2026

Auditor: Principal Security Architect & Penetration Tester

Classification: Academic Staging Security Audit

1. Vulnerability Summary Dashboard

Vulnerability ID	Vulnerability Category	Severity	CVSS v3.1	Affected Component	Exploitability	Remediation Effort
FEASIFY-SEC-01	Broken Access Control / IDOR	**High**	7.5	`PUT /api/proposals/:id/status`	Trivial (Unauthenticated HTTP PUT)	Low (Add `verifyToken`)
FEASIFY-SEC-02	Hardcoded Authorization Bypass	**High**	7.2	`ProtectedRoute.tsx` & `index.js`	Moderate (Register `chairperson@gmail.com`)	Low (Remove hardcoded string)
FEASIFY-SEC-03	Missing Firestore Security Rules	**High**	7.4	Repository Root (`firestore.rules`)	Moderate (Direct client SDK writes)	Medium (Deploy rule file)
FEASIFY-SEC-04	Permissive Wildcard CORS Policy	**Medium**	5.3	`feasify-backend/index.js` L133, 150	Low (Cross-origin embedding)	Low (Restrict allowed origins)
FEASIFY-SEC-05	IPv6 Rate Limit Bypass Risk	**Low**	3.7	`express-rate-limit` keyGenerator	Low (IPv6 address clustering)	Low (Use `ipKeyGenerator`)

2. Detailed Vulnerability Analyses

2.1 FEASIFY-SEC-01: Insecure Direct Object Reference (IDOR) & Unauthenticated Status Mutation

- **Affected File:** `feasify-backend/index.js` (Line 773)
- **Description:** The Express endpoint `PUT /api/proposals/:proposalId/status` updates proposal records in Cloud Firestore and triggers academic exemplar ingestion without requiring authentication. Unlike `/api/audit/logs` which enforces `verifyToken`, this status route allows any external caller to forge approval or rejection decisions.
- **Reproducible Evidence:**

```
curl -X PUT http://localhost:10000/api/proposals/test_proposal_123/status \
  -H "Content-Type: application/json" \
  -d '{"status":"Approved","adviserRemarks":"Bypassed approval"}'
```

Returns `HTTP 200 OK` with `{ "success": true, "status": "Approved" }`.

- **Potential Impact:** Malicious actors or students can prematurely approve their own unreviewed capstone feasibility studies.
- **Recommended Remediation:**

1. Add `verifyToken` middleware to the route.
2. Verify that `req.user.role` is `"Adviser"` or `"Chairperson"`.
3. Verify that the adviser is specifically assigned to the student's section or team.

2.2 FEASIFY-SEC-02: Hardcoded Chairperson Email Privilege Escalation

- **Affected Files:**
- `Feasify/src/components/ProtectedRoute.tsx` (Line 31):

```
if (user.email?.toLowerCase() === "chairperson@gmail.com") {
  setAuthState({ isLoading: false, isAuthenticated: true, role: "Chairperson" });
  return;
}
```

- `feasify-backend/index.js` (Line 97):

```
const isChairperson = email === "chairperson@gmail.com" || role === "Chairperson";
```

- **Description:** Both client and backend grant supreme administrative Chairperson privileges solely based on checking whether the user's email matches the hardcoded string `"chairperson@gmail.com"`.
- **Potential Impact:** If an unauthorized user registers the email `"chairperson@gmail.com"` on Firebase Auth (e.g., in a test environment or during open registration), they obtain full unvetted administrative control over all student and faculty records.
- **Recommended Remediation:** Remove hardcoded email comparisons. Base administrative access strictly on verified Firebase Custom Claims (`adminAuth.setCustomUserClaims`) or validated database role fields.

2.3 FEASIFY-SEC-03: Absence of Tracked Cloud Firestore Security Rules

- **Affected Location:** Repository root (No `firestore.rules` file found).
- **Description:** The frontend code executes direct document creations via `addDoc(collection(db, "proposals"))` and modifications via `updateDoc`. In the absence of strict database-level security rules committed in version control, security depends entirely on cloud console configurations. If rules are set to public test mode (`allow read, write: if true`), all client validation can be circumvented.
- **Recommended Remediation:** Author and commit a production `firestore.rules` file into the repository enforcing:
- Role-based read/write checks.
- Immutability of proposal counts per group.
- Verification that only assigned team leaders can write proposals.

2.4 FEASIFY-SEC-04: Permissive CORS Wildcard Configuration

- **Affected File:** `feasify-backend/index.js` (Line 133, 150)
- **Description:**

```
app.use(cors()); // Allows any origin '*'
const io = new Server(server, { cors: { origin: "*", credentials: true } });
```

Combining `origin: "*"` with `credentials: true` in WebSocket/Socket.IO connections violates modern CORS security standards and permits arbitrary web origins to establish socket listeners.

- **Recommended Remediation:** Explicitly configure allowed origins using environment variables:

```
const allowedOrigins = [process.env.CLIENT_URL || "http://localhost:5173"];
app.use(cors({ origin: allowedOrigins, credentials: true }));
```

2.5 FEASIFY-SEC-05: IPv6 Rate Limiter Key Generation Warning

- **Affected File:** `feasify-backend/index.js` (Line 119, 127)
- **Description:** `express-rate-limit` throws `ERR_ERL_KEY_GEN_IPV6` because `req.ip` is used directly in custom `keyGenerator` without normalizing IPv6 subnets.
- **Recommended Remediation:** Use `ipKeyGenerator` helper or normalize `req.ip` for IPv6 addresses as recommended by `express-rate-limit` documentation.

---

3. Secret Safety & Repository Integrity

Check	Target	Status	Notes
Gitignore Coverage	`.gitignore`	**Passed**	`.env`, `node_modules`, and `.env.local` properly ignored
Backend Environment File	`feasify-backend/.env`	**Passed**	Contains template keys; production secrets not committed
Input Sanitization	`sanitizeUserFacingText()`	**Passed**	Strips internal prompt injection codes and criteria tokens
SQL Injection Risk	Entire Codebase	**N/A**	Cloud Firestore NoSQL is used; parameter concatenation not present
XSS Prevention	React 19 JSX	**Passed**	React auto-escapes rendered strings; `dangerouslySetInnerHTML` not misused